MULTIMODAL AI CHATBOT

Python-Based Project Report

Project	Multimodel AI Chat Bot (Study Buddy Chat)
Technology	Python · Streamlit · FastAPI · Google Gemini
Report date	30 October 2026
Project type	Module 3 — Multimodal Chatbot Lab

Prepared as a technical overview of the supplied Python project. Live provider verification and manual end-to-end checks are identified separately from automated test evidence.

Project Report

Project identity: **Multimodel AI Chat Bot** (application title: Study Buddy Chat).

1. Abstract

This project implements a Python-only multimodal chatbot that allows a user to interact with an AI model through text, images, and audio clips. The application is organized as a Streamlit user interface, a FastAPI backend, and a provider integration module that communicates with Google Gemini using the official Google GenAI Python SDK.

The backend validates uploaded media before sending it to the model. The application provides conversation-style output, attachment previews, loading feedback, readable errors, and a retry path. Conversation state is held in Streamlit session state; the project does not use a database or persistent media storage. The project is intended as an educational demonstration, not a production service.

2. Introduction

Traditional chatbots primarily accept text. A multimodal chatbot extends this interaction by allowing users to submit different forms of input, such as a written question, a photograph, or an audio recording. This project demonstrates a simple, unified chat experience for these three input types.

The implementation follows the supplied Module 3 Spec-First Multimodal Chatbot Lab brief. It uses Python for both the frontend and backend; it does not require a custom JavaScript, HTML, CSS, or Node.js application.

3. Problem Statement

Users may want to ask questions about information that is not conveniently represented as typed text. For example, a learner may want an explanation of a diagram, a description of a photograph, or a concise summary of a spoken recording. The project addresses this need through one interface that routes text, image, and audio inputs to a multimodal model and displays a text response.

4. Objectives

• Build one chat interface supporting text, image, and audio input.

• Connect the application to a multimodal AI model through a server-side integration.

• Keep the provider API key out of frontend code and browser-facing responses.

• Validate uploaded files by size, declared MIME type, extension, and media content checks where implemented.

• Display loading, success, and understandable error feedback.

• Keep the first version simple and stateless, with no user accounts or database.

• Provide setup instructions and a repeatable test checklist.

5. Scope of the Project

Included: text prompts; one image attachment with an optional question; one audio attachment with an optional instruction; text answers from Gemini; image/audio previews; upload validation; a health endpoint; and automated backend tests using mocked provider responses.

Not included: live voice conversations, video input, user accounts, database-backed history, long-term media storage, deployment, multi-user administration, or production-grade monitoring and abuse prevention.

6. Technology Stack

Python 3.10 or newer is specified by the project README. Streamlit provides the user interface. FastAPI exposes the backend endpoints. The Google GenAI Python SDK connects to Gemini. Requests is used by the Streamlit frontend to call the backend; Pillow is used to inspect image content; python-dotenv loads local environment configuration; and pytest is used for automated tests.

7. System Architecture

The system uses a three-layer design. The Streamlit frontend collects user input and displays messages. The FastAPI backend receives the request, checks the modality and uploaded file, enforces the configured size limit, and calls the provider module. The provider module builds the Gemini request and normalizes the model output into a text answer.

Data flow: User → Streamlit UI → FastAPI /api/chat → validation → Gemini provider module → Gemini model → normalized text response → Streamlit conversation display.

The API key is read by the backend from the local environment. The frontend communicates with the backend through the BACKEND_URL setting and does not contain the Gemini credential.

8. Main Modules

8.1 frontend/app.py — Streamlit chat interface. It offers a modality selector, text area, image/audio upload controls, image/audio previews, conversation display, a clear-conversation action, and visible loading/error feedback.

8.2 backend/main.py — FastAPI application. It defines /api/health and /api/chat, validates required input, checks upload size, verifies supported image/audio types, and maps provider configuration or execution failures to HTTP errors.

8.3 backend/provider.py — Gemini integration. Provider-specific SDK usage and model invocation are isolated here. The configured model defaults to gemini-2.5-flash. Provider errors are converted to user-readable messages rather than exposing SDK internals.

8.4 backend/config.py — Reads environment configuration such as GEMINI_API_KEY, GEMINI_MODEL, and MAX_UPLOAD_MB.

8.5 tests/test_api.py — Automated API tests. Provider behavior is mocked so the tests can run without a live Gemini key.

8.6 docs/MANUAL_TESTS.md — Manual checklist for startup, text/image/audio interactions, invalid uploads, service failure, secret handling, and keyboard navigation.

9. Functional Requirements

Text chat: a non-empty text prompt is submitted and the user/assistant messages are shown in the conversation.

Image understanding: a supported JPG, JPEG, PNG, or WebP image can be attached with an optional question. The backend checks the image format and content before provider submission.

Audio understanding: supported audio formats include WAV, MP3, OGG, WebM, M4A/MP4, AAC, and FLAC, subject to actual provider/model support. The backend checks the declared type, extension, and selected file signatures.

Feedback and retry: the interface shows a thinking indicator and readable error messages. Text input is retained on request failure so the user can retry.

Health check: GET /api/health reports service status, whether the provider key is configured, and the configured upload limit.

10. Non-Functional Requirements and Security

Secret management: GEMINI_API_KEY is loaded on the backend from a local .env file. The .env file is excluded from version control through .gitignore; the repository contains only .env.example.

Upload validation: the default maximum upload size is 10 MB. Unsupported types, empty files, oversized files, invalid image data, and mismatched audio signatures are rejected before the model call.

Privacy: uploaded media is read in memory and sent to Google Gemini for inference. The project does not save uploaded media to a local uploads folder or database. Provider-side data handling still applies, so only non-sensitive sample media should be used.

Usability: Streamlit's native controls provide labels and keyboard navigation. The interface includes attachment previews and visible status messages.

Limitations: the application is a local educational demo. It does not yet include authentication, rate limiting, HTTPS deployment configuration, monitoring, persistent conversation storage, or a formal privacy/retention layer. The provider SDK call is synchronous within the API request, and production deployment would need stronger timeout and concurrency controls.

11. Installation and Execution

Prerequisites: Python 3.10+, internet access, and a Gemini API key for live model responses.

Create and activate a virtual environment, then install dependencies with: pip install -r requirements.txt.

Copy .env.example to .env and set GEMINI_API_KEY. Do not share the key or commit the .env file.

Start the backend in the first terminal: uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000.

Start the frontend in a second terminal: streamlit run frontend/app.py.

Open the Streamlit URL printed in the terminal (normally http://localhost:8501). The backend health endpoint is http://127.0.0.1:8000/api/health and API documentation is available at http://127.0.0.1:8000/docs.

12. Testing and Verification

Automated test execution was performed from the project directory using: python -m pytest -q. Result: 7 tests passed in 0.45 seconds in the available execution environment.

The plain pytest -q command did not collect tests in this environment because the project package was not on the import path; using python -m pytest -q resolved test discovery/import context and passed all seven tests.

The automated tests use mocked provider responses. Therefore, this result verifies the tested backend behavior, not live Gemini connectivity or the quality of real image/audio answers.

Manual verification remains necessary for live text, image, and audio interactions; unsupported/corrupted and oversized files; provider failure and retry; duplicate submission behavior; keyboard-only navigation; and checking that no secret appears in frontend code or responses. The included docs/MANUAL_TESTS.md provides the checklist. These manual checks are not claimed as completed in this report.

13. Results

The project source includes the three input modalities, a Python-only UI/backend architecture, backend-side provider integration, media validation, local environment-based secret configuration, and automated API tests. The executed automated test suite completed with seven passing tests. A live provider test was not performed as part of report preparation because it requires the user's own valid Gemini API key and network access.

14. Limitations and Future Enhancements

Future improvements could include: explicit provider request timeouts and retry/backoff policy; request throttling and authentication before public deployment; stricter CORS and HTTPS configuration for hosted environments; more exhaustive media parser validation; a dedicated duplicate-submit lock; optional conversation export; richer accessibility testing; and a deployment guide.

Additional modalities such as live speech or video should be treated as separate scope extensions with their own acceptance criteria and tests.

15. Conclusion

The Multimodal AI Chatbot demonstrates how a single Python application can accept text, image, and audio input and return model-generated text. Its separation of Streamlit UI, FastAPI request handling, and Gemini provider integration keeps the code understandable and easier to maintain. The backend validation and server-side secret handling address important requirements from the lab brief. Automated tests pass, while live provider and end-to-end manual checks should be completed with a configured API key before final academic submission.

Appendix A. Project File Structure

```
multimodel-ai-chat-bot/ (archive name)
■■■ backend/
■   ■■■ config.py
■   ■■■ main.py
■   ■■■ provider.py
■   ■■■ __init__.py
■■■ frontend/
■   ■■■ app.py
■   ■■■ __init__.py
■■■ tests/
■   ■■■ test_api.py
■■■ docs/
■   ■■■ MANUAL_TESTS.md
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ README.md
■■■ SPEC.md
```

Appendix B. Configuration Variables

GEMINI_API_KEY — required for live Gemini responses; secret value belongs only in the local backend .env file.

GEMINI_MODEL — model identifier; default is gemini-2.5-flash.

MAX_UPLOAD_MB — maximum media upload size in megabytes; default is 10.

BACKEND_URL — backend base URL used by Streamlit; default is http://127.0.0.1:8000.